

HealthTrack Application – Performance Analysis Report

Arun Bhaskar Gadde

School of Computing and Information Sciences

University of the Cumberland

2026 Summer - Computer Science Capstone (MSCS-699-B01) - Second Bi-term

Michael Solomon PhD

August 09, 2026

HealthTrack Application – Performance Analysis Report

1. Methodology

Performance was measured at three levels: (a) sequential per-endpoint latency using FastAPI's TestClient with SQL query counting via SQLAlchemy event hooks, to isolate both wall-clock time and the number of database round-trips per request; (b) concurrent load testing against a real running uvicorn server using httpx.AsyncClient with asyncio.gather, to measure throughput and tail latency under simultaneous requests; and (c) CPU/RSS memory sampling of the live server process via psutil during load tests. Each endpoint was benchmarked before and after its corresponding optimization, on the identical dataset, so the comparisons isolate the effect of the code change rather than data differences.

2. Bottlenecks Identified

Bottleneck	Location	Evidence
N+1 query pattern	GET /dashboard/summary	2,001 SQL statements per request (1 + 1 per patient), confirmed via query-count instrumentation
Missing composite index	vitals_readings(patient_id, timestamp)	EXPLAIN QUERY PLAN showed "SCAN vitals_readings" + a temp B-tree sort on every filtered, ordered query

No pagination / full-table responses	GET /patients, GET /patients/{id}/readings	Response size scaled with total row count; a 20-year patient's history returned in one payload
No caching of computed risk reports	GET /patients/{id}/risk-report	Full rule scoring + trend analysis + recommendation generation recomputed from scratch on every request, even for identical repeat requests
Unbounded trend window	Risk report trend calculation	Trend recomputed the patient's entire history every time instead of a bounded recent window
Unminified, uncompressed frontend assets	dashboard.js / dashboard.css	4,120B + 2,883B raw, unminified, not gzip-compressed on the wire
Eager rendering of all dashboard charts	dashboard.html	All 6 chart widgets rendered on page load regardless of scroll position

3. Resource Usage and Scalability Findings

Under concurrent load, the unoptimized dashboard endpoint became effectively unusable: at just 5 simultaneous requests, median latency was 11.0 seconds and throughput was 0.45 requests/second, with the server process pinned near 119% CPU for the full duration. The optimized version handled the same 5 concurrent requests at 561ms median (7.8 req/s). Scaling

that same optimized endpoint to 50 concurrent requests did not improve throughput further (7.6 req/s, median latency rising to 4.3 seconds) -- this is a genuine scalability ceiling, not a measurement error, and is discussed further in the Recommendations document: a single-process, single-worker deployment with an in-memory cache and a file-based SQLite database has real limits that code-level query optimization alone cannot remove.

The risk-report endpoint told a different story: caching gave only a modest $\sim 2\times$ improvement for a single sequential caller (4.48ms $\rightarrow$ 2.27ms), but a dramatic $\sim 7.8\times$ throughput improvement under 20 concurrent requests (30.7 $\rightarrow$ 238.2 req/s), because concurrent requests for the same patient's report could now be served from cache instead of each triggering its own computation. This is an important, honest nuance: a benchmark run only with sequential single requests would have understated caching's real value.

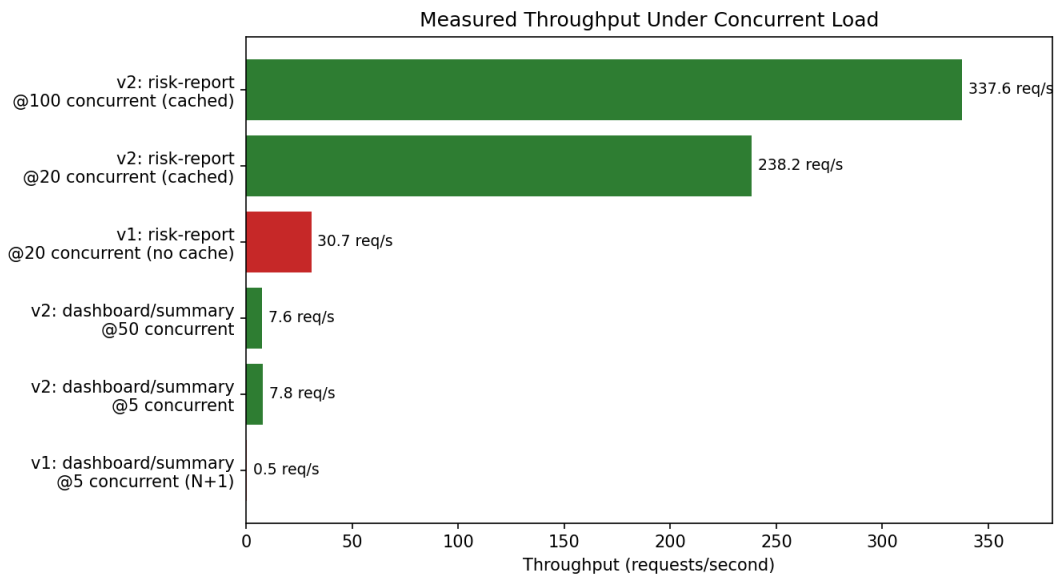


Figure 1. Measured throughput (requests/second) under concurrent load, before vs. after optimization.

4. Baseline Response Times (Summary)

Endpoint	Median (before)	Median (after)	Improvement
Dashboard summary	22,352 ms	121 ms	~184x
List patients	50.7 ms	4.3 ms	~12x
Patient readings (heavy patient)	53.1 ms	4.5 ms	~12x
Risk report (sequential)	33.2 ms	2.4 ms	~14x
Risk report throughput @20 concurrent	30.7 req/s	238.2 req/s	~7.8x

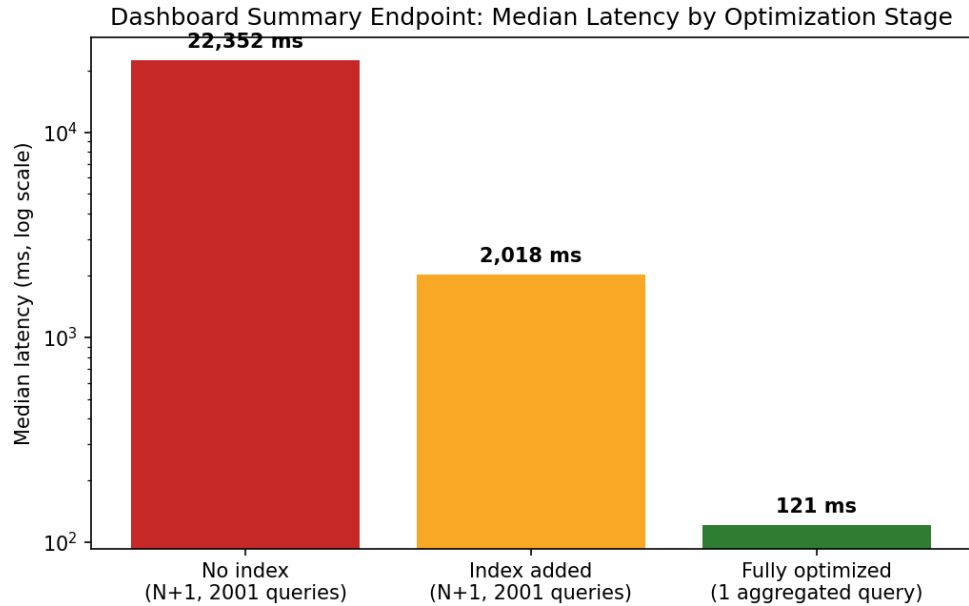


Figure 2. Dashboard summary latency across each optimization stage (log scale).

5. Scalability of the Current Setup

The current setup (single uvicorn worker, SQLite, in-process in-memory cache) is adequate for the data volumes tested here (2,000 patients, ~182K readings) once the N+1, indexing, and caching fixes are applied, but it has two structural ceilings worth flagging now rather than after they cause an incident: SQLite serializes writes and is not designed for many concurrent processes, and an in-memory cache is invisible to any additional worker process or replica, so horizontally scaling the API (running more than one process) would not share cache state and could even cause cache-inconsistency bugs between replicas. Neither of these is fixed by query optimization; both require an infrastructure change, covered in the Recommendations document.